

Chit 9

Collection "orderinfo" which contains the documents given as below(Perform on Mongo Terminal)

```
{
  cust_id:123
  cust_name:"abc",
  status:"A",
  price:250
}
```

- i. find the average price for each customers having status 'A'
- ii. Display the status of the customers whose amount/price lie between 100 and 1000
- iii. Display the customers information without "_id" .
- iv. create a simple index on orderinfo collection and fire the queries.

1. Create Database

use chit9

2. Create Collection and Insert Data

```
db.orderinfo.insertMany([
  { cust_id: 123, cust_name: "abc", status: "A", price: 250 },
  { cust_id: 124, cust_name: "xyz", status: "A", price: 500 },
  { cust_id: 125, cust_name: "pqr", status: "B", price: 150 },
  { cust_id: 126, cust_name: "lmn", status: "A", price: 800 }
])
```

3. (i) Average price for each customer with status 'A'

```
db.orderinfo.aggregate([
  { $match: { status: "A" } },
  {
    $group: {
      _id: "$cust_id",
      avgPrice: { $avg: "$price" }
    }
  }
])
```

4. (ii) Display status of customers whose price is between 100 and 1000

```
db.orderinfo.find(
  { price: { $gte: 100, $lte: 1000 } },
  { status: 1, _id: 0 }
)
```

5. (iii) Display customers info without _id

```
db.orderinfo.find({}, { _id: 0 })
```

6. (iv) Create a simple index and fire queries

Create index on **cust_id**:

```
db.orderinfo.createIndex({ cust_id: 1 })
```

Now run any query (example):

```
db.orderinfo.find({ cust_id: 123 })
```

Chit 12

Collection “movies“ which contains the documents given as below(Perform on Mongo Terminal)

```
{  
  name: "Movie1",  
  type: "action",  
  budget:1000000  
  producer:{  
    name: "producer1",  
    address:"PUNE"  
  }  
}
```

- i. Find the name of the movie having budget greater than 1,00,000.
- ii. Find the name of producer who lives in Pune
- iii. Update the type of movie “action” to “horror”
- iv. Find all the documents produced by name “producer1” with their address

1. Create Database

use chit12

2. Create Collection and Insert Document

```
db.movies.insertOne({  
  name: "Movie1",  
  type: "action",  
  budget: 1000000,  
  producer: {  
    name: "producer1",  
    address: "PUNE"  
  }  
})
```

3. (i) Find movie names with budget > 1,00,000

```
db.movies.find(  
  { budget: { $gt: 100000 } },  
  { name: 1, _id: 0 }  
)
```

4. (ii) Find producer name who lives in Pune

```
db.movies.find(  
  { "producer.address": "PUNE" },  
  { "producer.name": 1, _id: 0 }  
)
```

5. (iii) Update type from "action" to "horror"

```
db.movies.updateMany(  
  { type: "action" },  
  { $set: { type: "horror" } }  
)
```

6. (iv) Find all documents produced by "producer1" with address

```
db.movies.find(  
  { "producer.name": "producer1" },  
  { name: 1, "producer.address": 1, _id: 0 }  
)
```

Chit 13

Consider following structure for MongoDB collection and write a query for following requirements in MongoDB

Teachers (Tname,dno,Experience,Salary,Date_of_Joining)

Department (Dno,Dname)

Students(Sname,Roll_No,Class)

1. Write a query to create above collection insert some sample documents.
2. Find the information about all teachers of Dno=2 and having salary greater than or equal to 10,000/-
3. Find the student information having Roll_no=2 or Sname='xyz'
4. Update student name whose Roll_No=5
5. Delete all student whose Class is 'FE'
6. Apply index on Students Collection

1. Create Database

use chit13

2. Create Collections and Insert Sample Data

Teachers

```
db.teachers.insertMany([
  { Tname: "Amit", dno: 1, Experience: 5, Salary: 12000, Date_of_Joining: "2020-01-10" },
  { Tname: "Riya", dno: 2, Experience: 3, Salary: 10000, Date_of_Joining: "2021-06-15" },
  { Tname: "John", dno: 2, Experience: 7, Salary: 15000, Date_of_Joining: "2018-03-20" }
])
```

Department

```
db.department.insertMany([
  { Dno: 1, Dname: "IT" },
  { Dno: 2, Dname: "CS" }
])
```

Students

```
db.students.insertMany([
  { Sname: "Rahul", Roll_No: 1, Class: "SE" },
  { Sname: "xyz", Roll_No: 2, Class: "TE" },
  { Sname: "Priya", Roll_No: 5, Class: "FE" },
  { Sname: "Ankit", Roll_No: 6, Class: "FE" }
])
```

3. (i) Teachers of dno = 2 and salary ≥ 10000

```
db.teachers.find(
  { dno: 2, Salary: { $gte: 10000 } }
)
```

4. (ii) Student with Roll_No = 2 OR Sname = "xyz"

```
db.students.find(
  { $or: [ { Roll_No: 2 }, { Sname: "xyz" } ] }
)
```

5. (iii) Update student name whose Roll_No = 5

```
db.students.updateOne(
  { Roll_No: 5 },
  { $set: { Sname: "Neha" } }
)
```

6. (iv) Delete all students whose Class is "FE"

```
db.students.deleteMany(  
  { Class: "FE" }  
)
```

7. (v) Apply index on Students collection

```
db.students.createIndex({ Roll_No: 1 })
```

Chit 14

Use MongoDB

Create Institute Database and Create Student collection

1. RollNo 2. Student Name 3. Age 4. Branch 5. Address :{ City, State} 6. Hobbies (Array)

Perform following operations

1. Create database Institute.
2. Create collection Students.
3. Insert 10 documents with above mentioned structure.
4. Display all students' information.
5. Display Student information whose age is greater than 15.
6. Display Student information sorted on name field
7. Update student branch Computer of RollNo 3.
8. Remove document with RollNo 1
9. Display Student information whose name starts with A
10. Display the total numbers of documents available in collection.
11. Display only first 5 documents.
12. Display all documents instead of first 3.
13. Display the name of Students who live in Pune City.
14. Display only Name of all students.
15. Drop Collection

1. Create Database

use Institute

2. Create Collection

```
db.createCollection("students")
```

3. Insert 10 Documents

```
db.students.insertMany([
{ RollNo:1, Sname:"Amit", Age:18, Branch:"IT", Address:{City:"Pune", State:"MH"},
Hobbies:["Cricket","Music"] },
{ RollNo:2, Sname:"Riya", Age:17, Branch:"CS", Address:{City:"Mumbai", State:"MH"},
Hobbies:["Dance"] },
{ RollNo:3, Sname:"John", Age:19, Branch:"ENTC", Address:{City:"Pune", State:"MH"},
Hobbies:["Reading"] },
{ RollNo:4, Sname:"Ankit", Age:16, Branch:"IT", Address:{City:"Delhi", State:"DL"},
Hobbies:["Gaming"] },
{ RollNo:5, Sname:"Priya", Age:20, Branch:"CS", Address:{City:"Pune", State:"MH"},
Hobbies:["Music","Art"] },
{ RollNo:6, Sname:"Aryan", Age:18, Branch:"ME", Address:{City:"Nashik", State:"MH"},
Hobbies:["Sports"] },
{ RollNo:7, Sname:"Asha", Age:17, Branch:"IT", Address:{City:"Pune", State:"MH"},
Hobbies:["Dance"] },
{ RollNo:8, Sname:"Neha", Age:21, Branch:"CS", Address:{City:"Mumbai", State:"MH"},
Hobbies:["Reading"] },
{ RollNo:9, Sname:"Karan", Age:19, Branch:"ENTC", Address:{City:"Pune", State:"MH"},
Hobbies:["Gaming"] },
{ RollNo:10, Sname:"Alok", Age:22, Branch:"IT", Address:{City:"Delhi", State:"DL"},
Hobbies:["Music"] }
])
```

4. Display all students

```
db.students.find()
```

5. Students with age > 15

```
db.students.find({ Age: { $gt: 15 } })
```

6. Sort by name

```
db.students.find().sort({ Sname: 1 })
```

7. Update branch of RollNo 3 to "Computer"

```
db.students.updateOne(  
  { RollNo: 3 },  
  { $set: { Branch: "Computer" } }  
)
```

8. Remove document with RollNo 1

```
db.students.deleteOne({ RollNo: 1 })
```

9. Names starting with A

```
db.students.find({ Sname: /^A/ })
```

10. Total number of documents

```
db.students.countDocuments()
```

11. Display first 5 documents

```
db.students.find().limit(5)
```

12. Display all documents except first 3

```
db.students.find().skip(3)
```

13. Students living in Pune

```
db.students.find(  
  { "Address.City": "Pune" },  
  { Sname: 1, _id: 0 }  
)
```

14. Display only names

```
db.students.find({}, { Sname: 1, _id: 0 })
```

15. Drop Collection

```
db.students.drop()
```

Chit 15

Perform aggregation and Indexing using mongodb on below database

1. Create a database department
2. Create a collection as teacher with fields as name , department ,experience and salary
3. Display the department wise average salary.
4. Display the no. Of employees working in each department.
5. Display the department wise minimum salary.
6. Apply index and drop index

1. Create Database

use department

2. Create Collection and Insert Data

```
db.teacher.insertMany([
  { name: "Amit", department: "IT", experience: 5, salary: 20000 },
  { name: "Riya", department: "CS", experience: 3, salary: 15000 },
  { name: "John", department: "IT", experience: 7, salary: 30000 },
  { name: "Neha", department: "CS", experience: 4, salary: 18000 },
  { name: "Karan", department: "ENTC", experience: 6, salary: 25000 }
])
```

3. Department-wise Average Salary

```
db.teacher.aggregate([
  {
    $group: {
      _id: "$department",
      avgSalary: { $avg: "$salary" }
    }
  }
])
```

4. Number of Employees in Each Department

```
db.teacher.aggregate([
  {
    $group: {
      _id: "$department",
      totalEmployees: { $sum: 1 }
    }
  }
])
```

5. Department-wise Minimum Salary

```
db.teacher.aggregate([
  {
    $group: {
      _id: "$department",
      minSalary: { $min: "$salary" }
    }
  }
])
```

Chit 18

Collection “orderinfo“ which contains the documents given as below(Perform on Mongo Terminal)

```
{
  cust_id:123
  cust_name:"abc",
  status:"A",
  price:250
}
```

- i. Add “Age” field to the orderinfo collection
- ii. Create the complex index on orderinfo collection and fire the queries and drop the duplicates.
- iii. Display the average price for each customer group by status
- iv. Change the customer’s name whose status is “B”

1. Create Database

use chit18

2. Create Collection and Insert Data

```
db.orderinfo.insertMany([
  { cust_id: 123, cust_name: "abc", status: "A", price: 250 },
  { cust_id: 124, cust_name: "xyz", status: "B", price: 500 },
  { cust_id: 125, cust_name: "pqr", status: "A", price: 300 },
  { cust_id: 126, cust_name: "lmn", status: "B", price: 700 }
])
```

3. (i) Add “Age” field

```
db.orderinfo.updateMany(
  {},
  { $set: { Age: 25 } }
)
```

4. (ii) Create complex index and drop duplicates

Create compound (complex) index:

```
db.orderinfo.createIndex({ cust_id: 1, status: 1 })
```

To remove duplicates based on **cust_id**:

```
db.orderinfo.aggregate([
  {
    $group: {
      _id: "$cust_id",
      ids: { $push: "$_id" },
      count: { $sum: 1 }
    }
  },
  { $match: { count: { $gt: 1 } } }
]).forEach(doc => {
  doc.ids.shift();
  db.orderinfo.deleteMany({ _id: { $in: doc.ids } });
})
```


5. (iii) Average price for each customer grouped by status

```
db.orderinfo.aggregate([
  {
    $group: {
      _id: "$status",
      avgPrice: { $avg: "$price" }
    }
  }
])
```

6. (iv) Change customer name where status is "B"

```
db.orderinfo.updateMany(
  { status: "B" },
  { $set: { cust_name: "UpdatedName" } }
)
```

